

Waitlist (Chunk 2) — FB Landing Pages

Two public landing pages to convert FB ad traffic into typed leads while the lease marketplace is being built. **No auth.** Each page captures email plus a small profile (acres for landowners; states + budget + season interest for hunters) into a single `Waitlist` table.

Schema

Appended to `prisma/schema.prisma` :

- `model Waitlist` — see below
- `enum WaitlistSide { LANDOWNER HUNTER }`

Key columns on `Waitlist` :

Column	Type	Notes
id	String cuid	PK
side	WaitlistSide	LANDOWNER or HUNTER
email	String	normalized lowercase, validated by zod email
name	String?	optional first name
state	String?	landowner single-state code (e.g. MO)
states	String[]	hunter multi-state codes
acres	Float?	landowner only, > 0
maxBudgetUsd	Int?	hunter only, ≥ 0
seasonInterest	String[]	hunter chips: bow rifle muzzleloader youth
groupSize	Int?	hunter only, ≥ 1
source	String?	lease_your_land_landing / find_a_lease_landing
utmSource	String?	resolved from body / URL / referer (in that order)
utmMedium	String?	same
utmCampaign	String?	same
notes	String?	free text, max 2000 chars
createdAt	DateTime	

Indexes: (side, state) and (email) .

Idempotency: the API treats (email, side) as the dedupe key — a second submission for the same email + side updates the row in place rather than creating a duplicate.

OPSEC

The waitlistInputSchema is .strict() so any unknown field (e.g., centroidLat , lng , parcelId) is rejected with a 400 before it can hit the DB. Same hardening pattern as Listing .

Migration

Migrations directory now exists. Baseline `0_init` was created from the live schema and marked `applied`, then this migration was generated:

```
prisma/migrations/20260426141649_add_waitlist/migration.sql
```

Applying / regenerating:

```
cd nextjs_space
yarn prisma generate
yarn prisma migrate dev # picks up any pending migrations
```

API

- `POST /api/waitlist` — public, no auth.
- **Body:** depends on `side`. Common: `email`. Landowner: optional `name`, `state`, `acres`. Hunter: optional `name`, `states`, `maxBudgetUsd`, `seasonInterest`, `groupSize`. Both may include `notes`, `source`, `utmSource`, `utmMedium`, `utmCampaign`.
- `.strict()` — unknown fields → 400.
- **UTM resolution order:** body → request URL query → `Referer` header. First non-empty value wins per UTM key.
- **Idempotency:** if a row with the same `(email, side)` already exists, only non-null incoming fields overwrite, then **200** with `{ id, mode: "updated" }`.
- **New row:** **201** with `{ id, mode: "created" }`.
- **400** for validation errors with `{ error: "validation", details }`.
- **500** generic on unexpected DB issues.

No `GET` / `PATCH` / `DELETE` exposed publicly. Admin tooling will read this table directly via Prisma in chunk 3+.

Pages

- `/lease-your-land` — landowner-targeted. Hero IS the form (email / name / state / acres). Below the fold: 3-step how it works, a short why list with us deck, and FAQ. `source = "lease_your_land_landing"`.
- `/find-a-lease` — hunter-targeted. Hero IS the form (email / name / multi-state pills / max budget / season interest chips / group size). Below: certified-lease explainer, how it works, FAQ. `source = "find_a_lease_landing"`.

Both pages:

- Pull `utm_*` params off the URL via `useSearchParams()` and forward them in the POST body so referrer info survives client-side navigation.
- Fire GA4 `waitlist_join` event on success via `lib/gtag.ts` (already wired in `app/layout.tsx`).
- Show inline thank-you on success and an inline error message on 4xx/5xx.

- Use the same emerald/cream/stone palette as the rest of the marketing site so FB ad creative → landing → app feels continuous.

The forms live in `app/<page>/_form/*.tsx`. The `_form` and `_landing-shared` folders are excluded from routing by Next.js's `_` prefix convention.

Manual Test Steps

```
# Landowner happy path → 201
curl -i -X POST -H 'Content-Type: application/json' \
  -d '{"side":"LANDOWNER","email":"smoke-l@example.com","state":"MO","acres":120}' \
  https://terrafirma.partners/api/waitlist

# Same email + side → 200, mode: updated
curl -i -X POST -H 'Content-Type: application/json' \
  -d '{"side":"LANDOWNER","email":"smoke-l@example.com","state":"MO","acres":150}' \
  https://terrafirma.partners/api/waitlist

# Hunter happy path → 201
curl -i -X POST -H 'Content-Type: application/json' \
  -d '{"side":"HUNTER","email":"smoke-h@example.com","states":
["MO","KS"],"maxBudgetUsd":3000,"seasonInterest":["bow","rifle"]}' \
  https://terrafirma.partners/api/waitlist

# Unknown field → 400
curl -i -X POST -H 'Content-Type: application/json' \
  -d '{"side":"LANDOWNER","email":"x@x.com","centroidLat":12.3}' \
  https://terrafirma.partners/api/waitlist
```

Test Suite

```
cd nextjs_space
yarn test # vitest run
```

Current: 73/73 passing — listings (43) + waitlist (30: 19 zod + 11 API).

What's NOT in Chunk 2

- No admin dashboard for waitlist rows (read via Prisma directly for now).
- No double-opt-in email confirmation. Notify chunk 4.
- No segmentation export to CSV. Chunk 4.
- No public listings discovery — that's still chunk 3+.